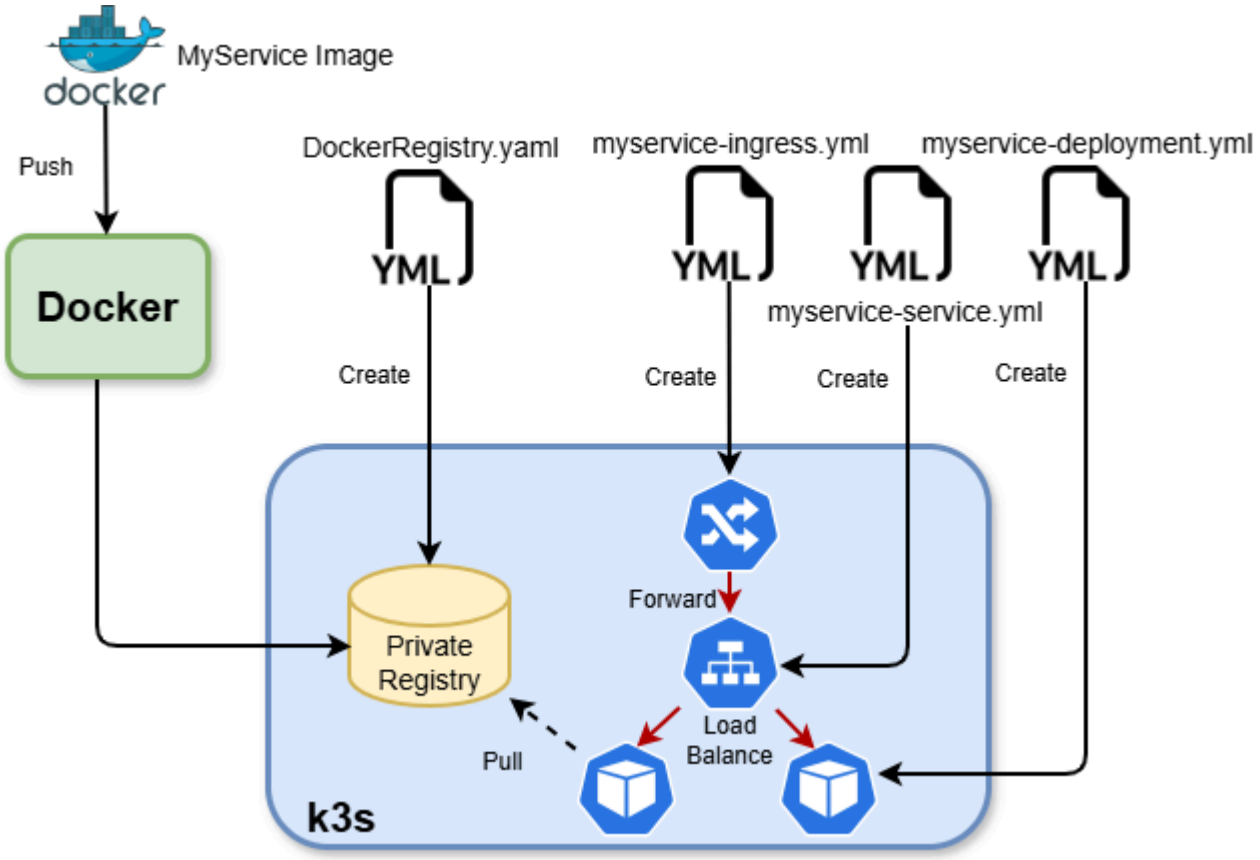
Presentation Schema



1. Service Containerization

- Create a Dockerfile for each of your REST and SOAP/gRPC services
- Create a docker-compose.yml gathering all your services
  - The name of the image parameter of the service within the docker-compose file should be of type : `image: registry.infres.fr/MyService` if the service is named MyService. This will allow easy push to the k3s cluster registry. i.e. :

```
version: '3.5'
services:
  node:
    image: registry.infres.fr/MyService
    build: .
    ports:
      - "MyPort:MyPort"
...
```

- Build your services and make them run on your local computer

2. k3S installation and configuration

k3s installation

Important : under Windows :

- Use wsl v2 updated to the latest version

```
wsl.exe --update
```

- Network mode should be set to `mirrored` in your .wslconfig file

Install & run k3s (K3S_KUBECONFIG_MODE="644" to allow non root user to read k3s.yaml file)

```
sudo su -
curl -sfL https://get.k3s.io | K3S_KUBECONFIG_MODE="644" sh -
```

Disable k3s at startup

```
systemctl disable k3s
```

Test that eveything is OK with a local user

```
k3s kubectl get node
```

To directly use kubectl CLI make KUBECONFIG points to k3s.yaml file (/etc/rancher/k3s/k3s.yaml on linux)

```
export KUBECONFIG="/etc/rancher/k3s/k3s.yaml"
kubectl get node
```

Registry installation

Make registry.infres.fr points to your local IP within your host resolution file

```
sudo su -
echo "$(hostname -I | awk '{print $1}') registry.infres.fr" >> /etc/hosts
```

Create a DockerRegistry.yaml file containing all necessary stuff to create a Docker registry

```
apiVersion: networking.k8s.io/v1
kind: Ingress
```

```
  metadata:
    name: docker-registry-ingress
    annotations:
      kubernetes.io/ingress.class: "traefik"
  spec:
    rules:
      - host: registry.infres.fr
        http:
          paths:
            - path: /
              pathType: Prefix
              backend:
                service:
                  name: docker-registry-service
                  port:
                    number: 5000
  ---
  apiVersion: v1
  kind: Service
  metadata:
    name: docker-registry-service
    labels:
      run: docker-registry
  spec:
    selector:
      app: docker-registry
    ports:
      - protocol: TCP
        port: 5000

  ---
  apiVersion: apps/v1
  kind: Deployment
  metadata:
    name: docker-registry
    labels:
      app: docker-registry
  spec:
    replicas: 1
    selector:
      matchLabels:
        app: docker-registry
    template:
      metadata:
        labels:
          app: docker-registry
      spec:
        containers:
          - name: docker-registry
            image: registry:2.8.2
            ports:
              - containerPort: 5000
                protocol: TCP
            volumeMounts:
```

```
            - name: storage
              mountPath: /var/lib/registry
          env:
            - name: REGISTRY_HTTP_ADDR
              value: :5000
            - name: REGISTRY_STORAGE_FILESYSTEM_ROOTDIRECTORY
              value: /var/lib/registry
      volumes:
        - name: storage
          emptyDir: {} # TODO -make this more permanent later
```

Deploy Docker registry directly on k3s cluster

```
kubectl apply -f DockerRegistry.yaml
```

Set registry configuration for k8s cluster (allow k3s cluster to pull your image)

```
sudo su -

cat <<EOF >/etc/rancher/k3s/registries.yaml
mirrors:
  registry.infres.fr:
    endpoint:
      - "http://registry.infres.fr"
EOF

systemctl restart k3s
```

Add registry configuration for local Docker engine (allow your local docker command to push your image)

```
sudo su -

// Edit and append to any existing configuration, the registry customization
vim /etc/docker/daemon.json
{
  ...
  "insecure-registries": [
    "registry.infres.fr"
  ]
}

systemctl restart docker
```

3. Build and push your server on the k3s registry

Build MyService service (from within its directory) and push it to the k3s registry

```
docker-compose build
docker-compose push
```

Test if the image has been correctly pushed on the registry

```
wget http://registry.infres.fr/v2/MyService/tags/list
```

4. Deploy a service named "MyService"

Make MyService.infres.fr.infres.fr points to your local IP within your host resolution file

```
echo "$(hostname -I | awk '{print $1}') MyService.infres.fr" >> /etc/hosts
```

Create a Kubernetes yaml file containing :

- An Ingress object allowing kubernetes to route a request coming from outside of the k8s vlan to the Service object of your pod

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: MyService-ingress
  annotations:
    kubernetes.io/ingress.class: "traefik"
spec:
  rules:
    - host: MyService.infres.fr
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: MyService-service
                port:
                  number: MyPort
```

- A Service object acting as a persistent Virtual IP that load-balances requests to the different replicas of your pod

```
apiVersion: v1
kind: Service
metadata:
  name: MyService-service
  labels:
    run: MyService
spec:
  selector:
    app: MyService
  ports:
    - protocol: TCP
      port: MyPort
```

- A Deployment object containing the description of your pod and the way to ensure its high availability (number of replicas needed etc.)

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: MyService
  labels:
    app: MyService
spec:
  replicas: 2
  selector:
    matchLabels:
      app: MyService
  template:
    metadata:
      labels:
        app: MyService
    spec:
      containers:
        - name: MyService
          image: registry.infres.fr/MyService
          imagePullPolicy: Always
          command: ["MyCommand,"MyParameter1","MyParameter2"...]
          ports:
            - containerPort: MyPort
```

Note : every variables starting with "My" should be replaced accordingly to your service

Deploy MyService on the cluster `kubectl apply -f MyService.yaml`

5. Add a hpa to your service and test it

Add a hpa file named MyService-hpa.yaml containing a description of how the number of replicas of your pod should scale up and down regarding CPU and RAM resources used by the pod

```
apiVersion: autoscaling/v2beta2
kind: HorizontalPodAutoscaler
metadata:
  name: MyService-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: MyService
  minReplicas: 2
  maxReplicas: 4
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 80
    - type: Resource
      resource:
        name: memory
        target:
          type: Utilization
          averageUtilization: 80
```

Deploy this hpa on the cluster `kubectl apply -f MyService-hpa.yaml`

Try to stress your service (CPU or RAM) in order to pass the threshold and trigger a pod scaling up and validate that the number of replicas increase accordingly.

Uninstall k3S

With root user :

`/usr/local/bin/k3s-uninstall.sh`